Review Questions

The answers to the chapter review questions can be found in the Appendix.

1. Which of the following Java operators can be used with `boolean` variables? (Choose all that apply.)
   - **A.** `==`
   - **B.** `+`
   - **C.** `--`
   - **D.** `!`
   - **E.** `%`
   - **F.** `~`
   - **G.** Cast with `(boolean)`

2. What data type (or types) will allow the following code snippet to compile? (Choose all that apply.)

```
byte apples = 5;
short oranges = 10;
_____ bananas = apples + oranges;
```

   - **A.** `int`
   - **B.** `long`
   - **C.** `boolean`
   - **D.** `double`
   - **E.** `short`
   - **F.** `byte`

3. What change, when applied independently, would allow the following code snippet to compile? (Choose all that apply.)

```
3: long ear = 10;
4: int hearing = 2 * ear;
```

   - **A.** No change; it compiles as is.
   - **B.** Cast `ear` on line 4 to `int`.
   - **C.** Change the data type of `ear` on line 3 to `short`.
   - **D.** Cast `2 * ear` on line 4 to `int`.
   - **E.** Change the data type of `hearing` on line 4 to `short`.
   - **F.** Change the data type of `hearing` on line 4 to `long`.

4. What is the output of the following code snippet?

```
3: boolean canine = true, wolf = true;
4: int teeth = 20;
5: canine = (teeth != 10) ^ (wolf=false);
6: System.out.println(canine+", "+teeth+", "+wolf);
```

A. `true, 20, true`
B. `true, 20, false`
C. `false, 10, true`
D. `false, 20, false`
E. The code will not compile because of line 5.
F. None of the above.

5. Which of the following operators are ranked in increasing or the same order of precedence? Assume the + operator is binary addition, not the unary form. (Choose all that apply.)

A. +, *, %, --
B. ++, (int), *
C. =, ==, !
D. (short), =, !, *
E. *, /, %, +, ==
F. !, ||, &
G. ^, +, =, +=

6. What is the output of the following program?

```
1: public class CandyCounter {
2:    static long addCandy(double fruit, float vegetables) {
3:       return (int)fruit+vegetables;
4:    }
5:
6:    public static void main(String[] args) {
7:       System.out.print(addCandy(1.4, 2.4f) + ", ");
8:       System.out.print(addCandy(1.9, (float)4) + ", ");
9:       System.out.print(addCandy((long)(int)(short)2, (float)4)); } }
```

A. `4, 6, 6.0`
B. `3, 5, 6`
C. `3, 6, 6`
D. `4, 5, 6`
E. The code does not compile because of line 9.
F. None of the above.

7. What is the output of the following code snippet?

```
int ph = 7, vis = 2;
boolean clear = vis > 1 & (vis < 9 || ph < 2);
boolean safe = (vis > 2) && (ph++ > 1);
boolean tasty = 7 <= --ph;
System.out.println(clear + "-" + safe + "-" + tasty);
```

- **A.** `true-true-true`
- **B.** `true-true-false`
- **C.** `true-false-true`
- **D.** `true-false-false`
- **E.** `false-true-true`
- **F.** `false-true-false`
- **G.** `false-false-true`
- **H.** `false-false-false`

8. What is the output of the following code snippet?

```
4: int pig = (short)4;
5: pig = pig++;
6: long goat = (int)2;
7: goat -= 1.0;
8: System.out.print(pig + " - " + goat);
```

- **A.** `4 - 1`
- **B.** `4 - 2`
- **C.** `5 - 1`
- **D.** `5 - 2`
- **E.** The code does not compile due to line 7.
- **F.** None of the above.

9. What are the unique outputs of the following code snippet? (Choose all that apply.)

```
int a = 2, b = 4, c = 2;
System.out.println(a > 2 ? --c : b++);
System.out.println(b = (a!=c ? a : b++));
System.out.println(a > b ? b < c ? b : 2 : 1);
```

- **A.** 1
- **B.** 2
- **C.** 3
- **D.** 4
- **E.** 5
- **F.** 6
- **G.** The code does not compile.

10. What are the unique outputs of the following code snippet? (Choose all that apply.)

```
short height = 1, weight = 3;
short zebra = (byte) weight * (byte) height;
double ox = 1 + height * 2 + weight;
long giraffe = 1 + 9 % height + 1;
System.out.println(zebra);
System.out.println(ox);
System.out.println(giraffe);
```

A. `1`

B. `2`

C. `3`

D. `4`

E. `5`

F. `6`

G. The code does not compile.

11. What is the output of the following code?

```
11: int sample1 = (2 * 4) % 3;
12: int sample2 = 3 * 2 % 3;
13: int sample3 = 5 * (1 % 2);
14: System.out.println(sample1 + ", " + sample2 + ", " + sample3);
```

A. `0, 0, 5`

B. `1, 2, 10`

C. `2, 1, 5`

D. `2, 0, 5`

E. `3, 1, 10`

F. `3, 2, 6`

G. The code does not compile.

12. The _________ operator increases a value and returns the original value, while the _______ operator decreases a value and returns the new value.

A. post-increment, post-increment

B. pre-decrement, post-decrement

C. post-increment, post-decrement

D. post-increment, pre-decrement

E. pre-increment, pre-decrement

F. pre-increment, post-decrement

13. What is the output of the following code snippet?

```
boolean sunny = true, raining = false, sunday = true;
boolean goingToTheStore = sunny & raining ^ sunday;
boolean goingToTheZoo = sunday && !raining;
boolean stayingHome = !(goingToTheStore && goingToTheZoo);
System.out.println(goingToTheStore + "-" + goingToTheZoo
   + "-" +stayingHome);
```

A. `true-false-false`

B. `false-true-false`

C. `true-true-true`

D. `false-true-true`

E. `false-false-false`

F. `true-true-false`

G. None of the above

14. Which of the following statements are correct? (Choose all that apply.)

A. The return value of an assignment operation expression can be `void`.

B. The inequality operator (`!=`) can be used to compare objects.

C. The equality operator (`==`) can be used to compare a `boolean` value with a numeric value.

D. During runtime, the & and | operators may cause only the left side of the expression to be evaluated.

E. The return value of an assignment operation expression is the value of the newly assigned variable.

F. In Java, `0` and `false` may be used interchangeably.

G. The logical complement operator (`!`) cannot be used to flip numeric values.

15. Which operators take three operands or values? (Choose all that apply.)

A. `=`

B. `&&`

C. `*=`

D. `? :`

E. `&`

F. `++`

G. `/`

16. How many lines of the following code contain compiler errors?

```
int note = 1 * 2 + (long)3;
short melody = (byte)(double)(note *= 2);
double song = melody;
float symphony = (float)((song == 1_000f) ? song * 2L : song);
```

A. 0

B. 1

C. 2

D. 3

E. 4

17. Given the following code snippet, what are the values of the variables after it is executed? (Choose all that apply.)

```
int ticketsTaken = 1;
int ticketsSold = 3;
ticketsSold += 1 + ticketsTaken++;
ticketsTaken *= 2;
ticketsSold += (long)1;
```

A. `ticketsSold` is 8.

B. `ticketsTaken` is 2.

C. `ticketsSold` is 6.

D. `ticketsTaken` is 6.

E. `ticketsSold` is 7.

F. `ticketsTaken` is 4.

G. The code does not compile.

18. Which of the following can be used to change the order of operation in an expression? (Choose all that apply.)

A. `[ ]`

B. `< >`

C. `( )`

D. `\ /`

E. `{ }`

F. `" "`

19. What is the result of executing the following code snippet? (Choose all that apply.)

```
3: int start = 7;
4: int end = 4;
5: end += ++start;
6: start = (byte)(Byte.MAX_VALUE + 1);
```

A. `start` is `0`.
B. `start` is `-128`.
C. `start` is `127`.
D. `end` is `8`.
E. `end` is `11`.
F. `end` is `12`.
G. The code does not compile.
H. The code compiles but throws an exception at runtime.

20. Which of the following statements about unary operators are true? (Choose all that apply.)

A. Unary operators are always executed before any surrounding numeric binary or ternary operators.
B. The `-` operator can be used to flip a `boolean` value.
C. The pre-increment operator (`++`) returns the value of the variable before the increment is applied.
D. The post-decrement operator (`--`) returns the value of the variable before the decrement is applied.
E. The `!` operator cannot be used on numeric values.
F. None of the above

21. What is the result of executing the following code snippet?

```
int myFavoriteNumber = 8;
int bird = ~myFavoriteNumber;
int plane = -myFavoriteNumber;
var superman = bird == plane ? 5 : 10;
System.out.println(bird + "," + plane + "," + --superman);
```

A. `-7,-8,9`
B. `-7,-8,10`
C. `-8,-8,4`
D. `-8,-8,5`
E. `-9,-8,9`
F. `-9,-8,10`
G. None of the above

Respuestas del cuestionario semana 2	
Pregunta	Comentario
1	Elegí A porque == sirve para comparar valores booleanos. Elegí D porque ! se usa para cambiar true por false y viceversa. También elegí G porque boolean es un tipo de dato y puede usarse en un cast.
2	Cuando se suman un byte y un short, Java convierte el resultado a int. Por eso se puede guardar en int, long o double. No se puede guardar directamente en byte o short sin hacer un cast.
3	El problema es que ear es un long, entonces 2 * ear también produce un long. No se puede guardar automáticamente en un int. Las opciones correctas son las que convierten el resultado a int o cambian el tipo de dato para que sea compatible.
4	teeth != 10 da true porque 20 es diferente de 10. La expresión (wolf = false) cambia el valor de wolf a false. Después true ^ false da true, así que canine termina siendo true.
5	Las opciones A y C muestran operadores ordenados correctamente según su precedencia. Las demás mezclan operadores de mayor y menor prioridad en un orden incorrecto.
6	El método debe devolver un long, pero la expresión (int)fruit + vegetables produce un float. Java no convierte automáticamente un float a long, por eso el programa no compila.
7	La primera expresión da true. La segunda da false porque vis > 2 es falso y el lado derecho no se evalúa. Luego --ph reduce el valor a 6 y 7 <= 6 da false.
8	pig++ devuelve primero el valor original y luego incrementa. Pero después se vuelve a asignar ese valor original a pig, por eso queda en 4. En goat -= 1.0, el valor final queda en 1.
9	Primero se imprime 4 porque se usa b++. Luego se imprime 5 porque el valor actual de b es 5. Finalmente se imprime 1 porque la condición a > b es falsa.
10	El código no compila porque al multiplicar dos valores byte, Java convierte el resultado a int. Después intenta guardar ese int en un short y eso genera un error.
11	Hice las operaciones siguiendo la precedencia. 8 % 3 da 2, 6 % 3 da 0 y 5 * (1 % 2) da 5.
12	x++ devuelve primero el valor original y luego aumenta. --x disminuye primero el valor y devuelve el nuevo resultado.
13	Primero obtuve true usando & y ^. Luego !raining da true, por lo que goingToTheZoo también es true. Finalmente !(true && true) da false.
14	Elegí B porque los objetos pueden compararse usando !=. Elegí E porque una asignación devuelve el valor que se asignó. Elegí G porque el operador ! solo funciona con valores booleanos.
15	El operador ternario usa tres partes: una condición, un valor para verdadero y otro para falso. Por eso es el único operador que usa tres operandos.
16	Solo la primera línea tiene error. El resultado de la expresión termina siendo un long y Java no permite guardarlo directamente en una variable int.

17	ticketsTaken++ usa primero el valor 1 y después aumenta a 2. Luego se multiplica por 2 y queda en 4. ticketsSold termina pasando de 3 a 5 y finalmente a 6.
18	Los paréntesis permiten cambiar el orden en que Java evalúa una expresión. Los demás símbolos no sirven para eso.
19	++start incrementa primero el valor y por eso end termina en 12. Después ocurre un overflow al convertir 128 a byte, y el resultado es -128.
20	Elegí A porque los operadores unarios tienen prioridad alta. Elegí D porque x-- devuelve primero el valor original. Elegí E porque ! solo puede usarse con booleanos.
21	~8 produce -9. El valor plane es -8 porque simplemente cambia el signo. Como no son iguales, el operador ternario devuelve 10. Luego --superman lo reduce a 9 antes de imprimirlo.

Review Questions

The answers to the chapter review questions can be found in the Appendix.

1. Which of the following data types can be used in a `switch` expression? (Choose all that apply.)

   A. `enum`
   B. `int`
   C. `Byte`
   D. `long`
   E. `String`
   F. `char`
   G. `var`
   H. `double`

2. What is the output of the following code snippet? (Choose all that apply.)

```
3: int temperature = 4;
4: long humidity = -temperature + temperature * 3;
5: if (temperature>=4)
6: if (humidity < 6) System.out.println("Too Low");
7: else System.out.println("Just Right");
8: else System.out.println("Too High");
```

   A. `Too Low`
   B. `Just Right`
   C. `Too High`
   D. A `NullPointerException` is thrown at runtime.
   E. The code will not compile because of line 7.
   F. The code will not compile because of line 8.

3. Which of the following data types are permitted on the right side of a for-each expression? (Choose all that apply.)

   A. `Double[][]`
   B. `Object`
   C. `Map`
   D. `List`
   E. `String`
   F. `char[]`
   G. `Exception`
   H. `Set`

4. What is the output of calling `printReptile(6)`?

```
void printReptile(int category) {
  var type = switch(category) {
      case 1,2 -> "Snake";
      case 3,4 -> "Lizard";
      case 5,6 -> "Turtle";
      case 7,8 -> "Alligator";
   };
   System.out.print(type);
}
```

   A. `Snake`
   B. `Lizard`
   C. `Turtle`
   D. `Alligator`
   E. `TurtleAlligator`
   F. None of the above

5. What is the output of the following code snippet?

```
List<Integer> myFavoriteNumbers = new ArrayList<>();
myFavoriteNumbers.add(10);
myFavoriteNumbers.add(14);
for (var a : myFavoriteNumbers) {
   System.out.print(a + ", ");
   break;
}

for (int b : myFavoriteNumbers) {
   continue;
   System.out.print(b + ", ");
}

for (Object c : myFavoriteNumbers)
   System.out.print(c + ", ");
```

   A. It compiles and runs without issue but does not produce any output.
   B. `10, 14,`
   C. `10, 10, 14,`
   D. `10, 10, 14, 10, 14,`
   E. Exactly one line of code does not compile.
   F. Exactly two lines of code do not compile.
   G. Three or more lines of code do not compile.
   H. The code contains an infinite loop and does not terminate.

6. Which statements about decision structures are true? (Choose all that apply.)

   A. A for-each loop can be executed on any Collections Framework object.

   B. The body of a `while` loop is guaranteed to be executed at least once.

   C. The conditional expression of a `for` loop is evaluated before the first execution of the loop body.

   D. A `switch` expression that takes a `String` and assigns the result to a variable requires a `default` branch.

   E. The body of a `do/while` loop is guaranteed to be executed at least once.

   F. An `if` statement can have multiple corresponding `else` statements.

7. Assuming `weather` is a well-formed nonempty array, which code snippet, when inserted independently into the blank in the following code, prints all of the elements of `weather`? (Choose all that apply.)

```
private void print(int[] weather) {
   for(____________________) {
      System.out.println(weather[i]);
   }
}
```

   A. `int i=weather.length; i>0; i--`

   B. `int i=0; i<=weather.length-1; ++i`

   C. `var w : weather`

   D. `int i=weather.length-1; i>=0; i--`

   E. `int i=0, int j=3; i<weather.length; ++i`

   F. `int i=0; ++i<10 && i<weather.length;`

   G. None of the above

8. What is the output of calling `printType(11)`?

```
void printType(Object o) {
   if(o instanceof Integer bat) {
      System.out.print("int");
   } else if(o instanceof Integer bat && bat < 10) {
      System.out.print("small int");
   } else if(o instanceof Long bat || bat <= 20) {
      System.out.print("long");
   } default {
      System.out.print("unknown");
   }
}
```

A. `int`

B. `small int`

C. `long`

D. `unknown`

E. Nothing is printed.

F. The code contains one line that does not compile.

G. The code contains two lines that do not compile.

H. None of the above

9. Which statements, when inserted independently into the following blank, will cause the code to print 2 at runtime? (Choose all that apply.)

```
int count = 0;
BUNNY: for(int row = 1; row <=3; row++)
   RABBIT: for(int col = 0; col <3 ; col++) {
      if((col + row) % 2 == 0)
         ____________________;
      count++;
   }
System.out.println(count);
```

A. `break BUNNY`

B. `break RABBIT`

C. `continue BUNNY`

D. `continue RABBIT`

E. `break`

F. `continue`

G. None of the above, as the code contains a compiler error.

10. Given the following method, how many lines contain compilation errors? (Choose all that apply.)

```
private DayOfWeek getWeekDay(int day, final int thursday) {
   int otherDay = day;
   int Sunday = 0;
   switch(otherDay) {
      default:
      case 1: continue;
      case thursday: return DayOfWeek.THURSDAY;
      case 2,10: break;
```

```
         case Sunday: return DayOfWeek.SUNDAY;
         case DayOfWeek.MONDAY: return DayOfWeek.MONDAY;
      }
      return DayOfWeek.FRIDAY;
   }
```

- **A.** None, the code compiles without issue.
- **B.** 1
- **C.** 2
- **D.** 3
- **E.** 4
- **F.** 5
- **G.** 6
- **H.** The code compiles but may produce an error at runtime.

11. What is the output of calling `printLocation(Animal.MAMMAL)`?

```
10: class Zoo {
11:    enum Animal {BIRD, FISH, MAMMAL}
12:    void printLocation(Animal a) {
13:       long type = switch(a) {
14:          case BIRD -> 1;
15:          case FISH -> 2;
16:          case MAMMAL -> 3;
17:          default -> 4;
18:       };
19:       System.out.print(type);
20:    } }
```

- **A.** 3
- **B.** 4
- **C.** 34
- **D.** The code does not compile because of line 13.
- **E.** The code does not compile because of line 17.
- **F.** None of the above

12. What is the result of the following code snippet?

```
int sing = 8, squawk = 2, notes = 0;
while(sing > squawk) {
   sing--;
   squawk += 2;
```

```
7:    notes += sing + squawk;
8: }
9: System.out.println(notes);
```

A. 11

B. 13

C. 23

D. 33

E. 50

F. The code will not compile because of line 7.

13. What is the output of the following code snippet?

```
2: boolean keepGoing = true;
3: int result = 15, meters = 10;
4: do {
5:    meters--;
6:    if(meters==8) keepGoing = false;
7:    result -= 2;
8: } while keepGoing;
9: System.out.println(result);
```

A. 7

B. 9

C. 10

D. 11

E. 15

F. The code will not compile because of line 6.

G. The code does not compile for a different reason.

14. Which statements about the following code snippet are correct? (Choose all that apply.)

```
for(var penguin : new int[2])
   System.out.println(penguin);
var ostrich = new Character[3];
for(var emu : ostrich)
   System.out.println(emu);
List<Integer> parrots = new ArrayList<Integer>();
for(var macaw  : parrots)
   System.out.println(macaw);
```

A. The data type of `penguin` is `Integer`.

B. The data type of `penguin` is `int`.

C. The data type of `emu` is undefined.

D. The data type of `emu` is `Character`.

E. The data type of `macaw` is `List`.

F. The data type of `macaw` is `Integer`.

G. None of the above, as the code does not compile.

15. What is the result of the following code snippet?

```
final char a = 'A', e = 'E';
char grade = 'B';
switch (grade) {
   default:
   case a:
   case 'B': 'C': System.out.print("great ");
   case 'D': System.out.print("good "); break;
   case e:
   case 'F': System.out.print("not good ");
}
```

A. `great`

B. `great good`

C. `good`

D. `not good`

E. The code does not compile because the data type of one or more `case` statements does not match the data type of the `switch` variable.

F. None of the above

16. Given the following array, which code snippets print the elements in reverse order from how they are declared? (Choose all that apply.)

```
char[] wolf = {'W', 'e', 'b', 'b', 'y'};
```

A.

```
int q = wolf.length;
for( ; ; ) {
   System.out.print(wolf[--q]);
   if(q==0) break;
}
```

B.

```
for(int m=wolf.length-1; m>=0; --m)
   System.out.print(wolf[m]);
```

C.

```
for(int z=0; z<wolf.length; z++)
   System.out.print(wolf[wolf.length-z]);
```

D.

```
int x = wolf.length-1;
for(int j=0; x>=0 && j==0; x--)
   System.out.print(wolf[x]);
```

E.

```
final int r = wolf.length;
for(int w = r-1; r>-1; w = r-1)
   System.out.print(wolf[w]);
```

F.

```
for(int i=wolf.length; i>0; --i)
   System.out.print(wolf[i]);
```

G. None of the above

17. What distinct numbers are printed when the following method is executed? (Choose all that apply.)

```
private void countAttendees() {
   int participants = 4, animals = 2, performers = -1;
   while((participants = participants+1) < 10) {}
   do {} while (animals++ <= 1);
   for( ; performers<2; performers+=2) {}

   System.out.println(participants);
   System.out.println(animals);
   System.out.println(performers);
}
```

A. 6

B. 3

C. 4

D. 5

E. 10

F. 9

G. The code does not compile.

H. None of the above

18. Which statements about pattern matching and flow scoping are correct? (Choose all that apply.)

A. Pattern matching with an `if` statement is implemented using the `instance` operator.

B. Pattern matching with an `if` statement is implemented using the `instanceon` operator.

C. Pattern matching with an `if` statement is implemented using the `instanceof` operator.

D. The pattern variable cannot be accessed after the `if` statement in which it is declared.

E. Flow scoping means a pattern variable is only accessible if the compiler can discern its type.

F. Pattern matching can be used to declare a variable with an `else` statement.

19. What is the output of the following code snippet?

```
double iguana = 0;
do {
   int snake = 1;
   System.out.print(snake++ + " ");
   iguana--;
} while (snake <= 5);
System.out.println(iguana);
```

A. `1 2 3 4 -4.0`

B. `1 2 3 4 -5.0`

C. `1 2 3 4 5 -4.0`

D. `0 1 2 3 4 5 -5.0`

E. The code does not compile.

F. The code compiles but produces an infinite loop at runtime.

G. None of the above

20. Which statements, when inserted into the following blanks, allow the code to compile and run without entering an infinite loop? (Choose all that apply.)

```
int height = 1;
L1: while(height++ <10) {
   long humidity = 12;
   L2: do {
      if(humidity-- % 12 == 0) _______________;
      int temperature = 30;
      L3: for( ; ; ) {
         temperature++;
         if(temperature>50) _______________;
      }
   } while (humidity > 4);
}
```

- **A.** `break L2` on line 8; `continue L2` on line 12
- **B.** `continue` on line 8; `continue` on line 12
- **C.** `break L3` on line 8; `break L1` on line 12
- **D.** `continue L2` on line 8; `continue L3` on line 12
- **E.** `continue L2` on line 8; `continue L2` on line 12
- **F.** None of the above, as the code contains a compiler error

21. A minimum of how many lines need to be corrected before the following method will compile?

```
void findZookeeper(Long id) {
   System.out.print(switch(id) {
      case 10 -> {"Jane"}
      case 20 -> {yield "Lisa";};
      case 30 -> "Kelly";
      case 30 -> "Sarah";
      default -> "Unassigned";
   });
}
```

- **A.** Zero
- **B.** One
- **C.** Two
- **D.** Three
- **E.** Four
- **F.** Five

22. What is the output of the following code snippet? (Choose all that apply.)

```
2: var tailFeathers = 3;
3: final var one = 1;
4: switch (tailFeathers) {
5:    case one: System.out.print(3 + " ");
6:    default: case 3: System.out.print(5 + " ");
7: }
8: while (tailFeathers > 1) {
9:    System.out.print(--tailFeathers + " "); }
```

- **A.** `3`
- **B.** `5 1`
- **C.** `5 2`
- **D.** `3 5 1`
- **E.** `5 2 1`
- **F.** The code will not compile because of lines 3–5.
- **G.** The code will not compile because of line 6.

23. What is the output of the following code snippet?

```
15: int penguin = 50, turtle = 75;
16: boolean older = penguin >= turtle;
17: if (older = true) System.out.println("Success");
18: else System.out.println("Failure");
19: else if(penguin != 50) System.out.println("Other");
```

A. `Success`
B. `Failure`
C. `Other`
D. The code will not compile because of line 17.
E. The code compiles but throws an exception at runtime.
F. None of the above

24. Which of the following are possible data types for `friends` that would allow the code to compile? (Choose all that apply.)

```
for(var friend in friends) {
   System.out.println(friend);
}
```

A. `Set`
B. `Map`
C. `String`
D. `int[]`
E. `Collection`
F. `StringBuilder`
G. None of the above

25. What is the output of the following code snippet?

```
String instrument = "violin";
final String CELLO = "cello";
String viola = "viola";
int p = -1;
switch(instrument) {
   case "bass" : break;
   case CELLO : p++;
   default: p++;
   case "VIOLIN": p++;
   case "viola" : ++p; break;
}
System.out.print(p);
```

A. -1
B. 0
C. 1
D. 2
E. 3
F. The code does not compile.

26. What is the output of the following code snippet? (Choose all that apply.)

```
9:  int w = 0, r = 1;
10: String name = "";
11: while(w < 2) {
12:    name += "A";
13:    do {
14:       name += "B";
15:       if(name.length()>0) name += "C";
16:       else break;
17:    } while (r <=1);
18:    r++; w++; }
19: System.out.println(name);
```

A. ABC
B. ABCABC
C. ABCABCABC
D. Line 15 contains a compilation error.
E. Line 18 contains a compilation error.
F. The code compiles but never terminates at runtime.
G. The code compiles but throws a `NullPointerException` at runtime.

27. What is printed by the following code snippet?

```
byte amphibian = 1;
String name = "Frog";
String color = switch(amphibian) {
   case 1 -> { yield "Red"; }
   case 2 -> { if(name.equals("Frog")) yield "Green"; }
   case 3 -> { yield "Purple"; }
   default -> throw new RuntimeException();
};
System.out.print(color);
```

A. `Red`

B. `Green`

C. `Purple`

D. `RedPurple`

E. An exception is thrown at runtime.

F. The code does not compile.

28. What is the output of calling `getFish("goldie")`?

```
void getFish(Object fish) {
   if (!(fish instanceof String guppy))
      System.out.print("Eat!");
   else if (!(fish instanceof String guppy)) {
      throw new RuntimeException();
   }
   System.out.print("Swim!");
}
```

A. `Eat!`

B. `Swim!`

C. `Eat!` followed by an exception.

D. `Eat!Swim!`

E. An exception is printed.

F. None of the above

29. What is the result of the following code?

```
1: public class PrintIntegers {
2:    public static void main(String[] args) {
3:       int y = -2;
4:       do System.out.print(++y + " ");
5:       while(y <= 5);
6: } }
```

A. `-2 -1 0 1 2 3 4 5`

B. `-2 -1 0 1 2 3 4`

C. `-1 0 1 2 3 4 5 6`

D. `-1 0 1 2 3 4 5`

E. The code will not compile because of line 5.

F. The code contains an infinite loop and does not terminate.

Respuestas del cuestionario semana 2 tema 3	
Pregunta	Comentario
1	Switch acepta enum, int, Byte, String, char y también var si resuelve a un tipo válido.
2	B. Imprime "Just Right" porque humidity vale 8 y entra al else del segundo if.
3	El for-each funciona con arreglos e implementaciones de Iterable como List y Set.
4	No compila porque el switch expression no cubre todos los posibles valores y no tiene default.
5	Hay una línea que no compila porque después de continue hay código inalcanzable.
6	El for evalúa la condición antes de ejecutar, switch expression necesita default en este caso y do/while ejecuta al menos una vez.
7	Ambas opciones recorren correctamente el arreglo sin errores.
8	Hay dos líneas que no compilan por problemas de alcance de variables y uso incorrecto de default.
9	Esas opciones hacen que count termine con valor 2.
10	Hay 4 errores de compilación en el método.
11	Imprime 3 porque Animal.MAMMAL retorna 3 en el switch expression.
12	El resultado final de notes es 23.
13	No compila porque faltan paréntesis en la condición del while.
14	Los tipos inferidos son int, Character e Integer.
15	No compila por un error de sintaxis en los case.
16	Son las opciones que imprimen el arreglo en orden inverso correctamente.
17	Los valores distintos impresos son 3 y 10.
18	Pattern matching usa instanceof y flow scoping controla dónde puede usarse la variable.
19	No compila porque snake está fuera de alcance en la condición while.
20	Son las únicas opciones que evitan el bucle infinito y permiten compilar.
21	Se necesitan corregir 4 líneas para que compile.
22	La salida es: 5 2 1.
23	Hay un else sin if correspondiente y el código no compila.
24	El código usa "in" en lugar de ":" y por eso no compila.
25	El valor final de p es 2.
26	El código entra en un bucle infinito porque r nunca cambia dentro del do/while.
27	No compila porque un caso del switch expression no devuelve valor en todos los caminos.
28	No compila porque la variable guppy se declara dos veces en el mismo alcance.
29	Imprime: -1 0 1 2 3 4 5 6.